Classes

1. What is a Class?

A **class is a blueprint** to create objects with shared structure + behavior.

It models **“what something is.”**

```
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }

  login() {
    console.log(`${this.name} logged in`);
  }
}
```

Usage

```
const user1 = new User("Prajwal", "prajwal@mail.com");
user1.login();
```

The Problem with Classes (At Scale)

Classes push you toward **Inheritance**.

Example:

```
class Admin extends User {
  deleteUser() {}
}

class SuperAdmin extends Admin {
  accessServer() {}
}
```

Now you get:

User → Admin → SuperAdmin → MegaAdmin → ...

This is called:

❌ **Inheritance Hell**

What is Composition?

Composition means:

Build objects by **combining small behaviors** instead of inheriting big ones.

Composition = "What something CAN DO"

Instead of:

Admin *is a* User

We say:

Admin *has abilities*:

- canLogin
- canDeleteUsers
- canAccessReports

4. Composition Example (Pure JS Style)

Step 1 — Create Behavior Functions

```
const canLogin = (state) => ({
  login: () => console.log(`${state.name} logged in`)
});

const canDeleteUsers = (state) => ({
  deleteUser: (id) => console.log(`Deleted user ${id}`)
});

const canViewReports = (state) => ({
  viewReports: () => console.log("Viewing reports")
});
```

Step 2 — Compose Them

```
const createAdmin = (name) => {
  const state = { name };

  return {
    ...state,
    ...canLogin(state),
    ...canDeleteUsers(state),
    ...canViewReports(state)
  };
};
```

Usage

```
const admin = createAdmin("Prajwal");

admin.login();
admin.deleteUser(10);
admin.viewReports();
```

5. Class-Based Thinking vs Composition Thinking

Class Thinking	Composition Thinking
"What is it?"	"What can it do?"
Rigid hierarchy	Flexible assembly
Inheritance	Behavior reuse
Tightly coupled	Loosely coupled
Hard to extend	Easy to mix features

8. When Should You Use Classes?

Classes are still good when:

✅ You need clear domain models (DTOs, Entities)
✅ Working with TypeScript OOP-heavy systems
✅ Libraries expect class instances
✅ Simple data modeling

🔹 9. When Should You Use Composition?

Use composition when:

✅ Building services
✅ Business logic layers
✅ React components
✅ Middleware
✅ Feature toggles
✅ Plugins / extensibility
✅ Large backend systems

Class-based design is like:

🏢 Building a fixed apartment tower.

Composition is like:

🧩 Designing modular smart homes you can rearrange anytime.

Modern software prefers **modularity over hierarchy**.

What is a Class in JavaScript?

Answer:

A class is syntactic sugar over JavaScript's prototype-based inheritance.
It provides a cleaner way to create constructor functions and define shared methods. Do you know JS is **not truly class-based** (it's prototype-based)?

Difference Between Instance Methods and Static Methods?

Instance Method	Static Method
Belongs to object	Belongs to class itself
Called via instance	Called via class
Uses instance data	Utility/helper logic

What is Composition?

Answer:

Composition is a design pattern where objects are built by combining reusable behaviors instead of inheriting from a hierarchy.

Build objects from capabilities, not ancestry.

Super Static and Extends

extends → creates inheritance
super → connects child to parent
static → belongs to the class itself (not instances)

1. extends — Creating a Relationship Between Classes

extends is used when one class wants to **reuse** another class's properties and methods.

👉 **Syntax**

```
class Child extends Parent {}
```

This means:

Child **inherits** everything from Parent.

```
class Vehicle {
  constructor(brand, speed) {
    this.brand = brand;
    this.speed = speed;
  }

  drive() {
    console.log(`${this.brand} is moving at ${this.speed} km/h`);
  }
}

class Car extends Vehicle {
  constructor(brand, speed, fuel) {
    super(brand, speed);
    this.fuel = fuel;
  }
}
```

✅ **Usage**

```
const car = new Car("Toyota", 80, 20);
car.drive();
```

What extends Actually Does (Internally)

JavaScript links prototypes like this:

Car.prototype → Vehicle.prototype → Object.prototype

So methods are **looked up the chain**.

This is called the **Prototype Chain**.

2. super — Talking to the Parent Class

super is used inside a child class to access the parent.

You use it in two places:

Where	Why
Constructor	To call parent constructor
Methods	To call parent methods

super() Inside Constructor

When you extend a class, you **must call super() before using this.**

```
class Car extends Vehicle {
  constructor(brand, speed, fuel) {
    super(brand, speed); // initializes parent
    this.fuel = fuel;
  }
}
```

If you don't call super(), JavaScript throws:

ReferenceError: Must call super constructor.

Using super to Call Parent Methods

```
class Car extends Vehicle {
  drive() {
    super.drive(); // call parent version
    console.log("Car-specific logic");
  }
}
```

Output

Toyota is moving at 80 km/h
Car-specific logic

Think of super As

“Go one level up and execute that logic.”

3. static — Belongs to the Class, Not the Object

Static methods are **not copied to instances**.

They belong directly to the class itself.

```
class MathUtil {
static add(a, b) {
return a + b;
}
}

MathUtil.add(2, 3); // ☑ works


const m = new MathUtil();
m.add(2, 3); // ❌ Error
```

Because static methods are **not instance methods**.

🔥 Why Static Exists

Use static when logic:

- Does not depend on object data
- Is utility/helper logic
- Should not require object creation

4. Static Properties

Static can also store shared data.

```
class Config {
  static appName = "Inventory App";
}

console.log(Config.appName);
```

All instances share this value.

5. Combining extends + super + static

```
class Employee {
  constructor(name) {
    this.name = name;
  }


  work() {
    console.log(`${this.name} is working`);
  }

  static company() {
    console.log("ABC Corp");
  }
}

class Manager extends Employee {
  constructor(name, teamSize) {
    super(name);
    this.teamSize = teamSize;
  }
```

```
  work() {
    super.work();
    console.log(`Managing team of ${this.teamSize}`);
  }
}


const m = new Manager("Prajwal", 5);

m.work();
Employee.company();
```

Why must super() be called first?

Because this is created by the parent constructor.
Until parent runs → this doesn't exist.

Are static methods inherited?

Yes.

Manager.company();

Static methods are also inherited through extends.

Can static methods access instance data?

No — they don't have this of an object.

Method Overriding in JavaScript (Classes)

Method Overriding happens when a **child class provides its own implementation** of a method that already exists in the parent class.

Same method name + different behavior in the derived class.

```
class Vehicle {
  drive() {
    console.log("Vehicle is driving");
  }
}

class Car extends Vehicle {
  drive() {   // Overriding parent method
    console.log("Car is driving smoothly");
  }
}

const c = new Car();
c.drive(); // Car is driving smoothly
```